Project - 1: Railway Gauges Data Analysis

This project demonstrates how to impact CSV data into Pandas, analyze it, and visualize results using Matplotlib.

Scenario 1: Basic Data Loading & Cleaning

Before analysis, we ensure the data is clean and computationally ready. This involves handling missing entries and ensuring numbers are treated as math-ready values rather than text.

Key Operations:

- **Loading:** Importing the CSV file into a structured environment (Pandas).
- **Imputation:** Replacing "NaN" or empty cells with `0` to prevent calculation errors.
- **Type Casting:** Converting Gauge measurements to numeric floats/integers.

```
#-------------------------------------------------------------------------------
#  Scenario-1--Basic Data Loading & Cleaning
# ------------------------------------------------------------------------------
# 1.1 Load the dataset
df = pd.read_csv('railway_gauges.csv')

# 1.2 Display first 5 rows
print("First 5 rows of the dataset:")
#view sample data
print(df.head())

# Display column names
print("\nColumn names in the dataset:")
#checks column
print(df.columns)
```

This is the "Discovery Phase" where we bring the file into our system and check its structure.
"pd.read_csv('railway_gauges.csv')"
"df.head()", This shows a "preview" of the first 5 rows of your data.
"df.columns",This generates a clean list of all the category names (Headers) in the file.

Data Cleaning Logic:
To ensure the analysis is 100% accurate,we applied two critical functions

1. **Filling gaps:**We used **.fillna(0)** to replace all missing entries with zeros.
2. **Numeric Conversion:** We used **to_numeric()** to convert all track measurements into math-ready values.

```python
# 1.3 Check for missing values
print("\nMissing values in each column:")
# missing values check
print(df.isnull().sum())

# Replace missing values with 0
df.fillna(0, inplace=True)

print("\nMissing values after replacing with 0:")
print(df.isnull().sum())
```

Result: The data is now clean, consistent, and ready for visualization.

```
Data types after conversion:
Year              str
Broad Gauge     int64
Metre Gauge     int64
Narrow Gauge    int64
Total           int64
dtype: object
PS C:\Users\DELL\Documents\Python\DataAnalytics\Proj
```

Scenario 2: Simple Visualization

In this stage, we turn thousands of numbers into a single, easy-to-read **Line Graph**. This allows the client to see the total expansion of the railway network over the years at a glance.

The Main Functions Used:

1. **data['Year'].str[:4] (The Slicer):** It takes a year like "1950-51" and cuts it down to just "1950."
2. **plt.plot() :**It draws the line connecting your data points.It reveals the "story" of the data—whether the trend is going up, down, or staying flat.
3. **plt.xlabel() & plt.ylabel() (The Labelers):**They add titles to the bottom and side of the graph.

Cleaning the Timeline:

In this step,we prepare data Since years in the file might look like "2010-11", we clean them so the computer sees them as a single number.

```python
#  Scenario-2--Simple Visualization
# ---------------------------------------------------------------------------
#2.1 Extract Year and Total columns
# 2.1 Select required columns (Year and Total tracks)
data = df[['Year', 'Total']].copy()

# 2.2 Extract year part + convert to numeric (cleaning )
data['Year'] = data['Year'].str[:4]
data['Year'] = pd.to_numeric(data['Year'], errors='coerce')

# convert Total to numeric (safe conversion)
data['Total'] = pd.to_numeric(data['Total'], errors='coerce')

# remove invalid/missing rows after conversion
data.dropna(inplace=True)
```

- **.str[:4]**: This acts like a pair of scissors. It cuts the long year text (like "1995-96") to keep only the first four digits ("1995").
- **.dropna()**: This is a "safety net." It removes any rows where the data is broken or missing so the graph doesn't look messy.

Now that the data is clean, we use a plotting tool to visualize the growth.

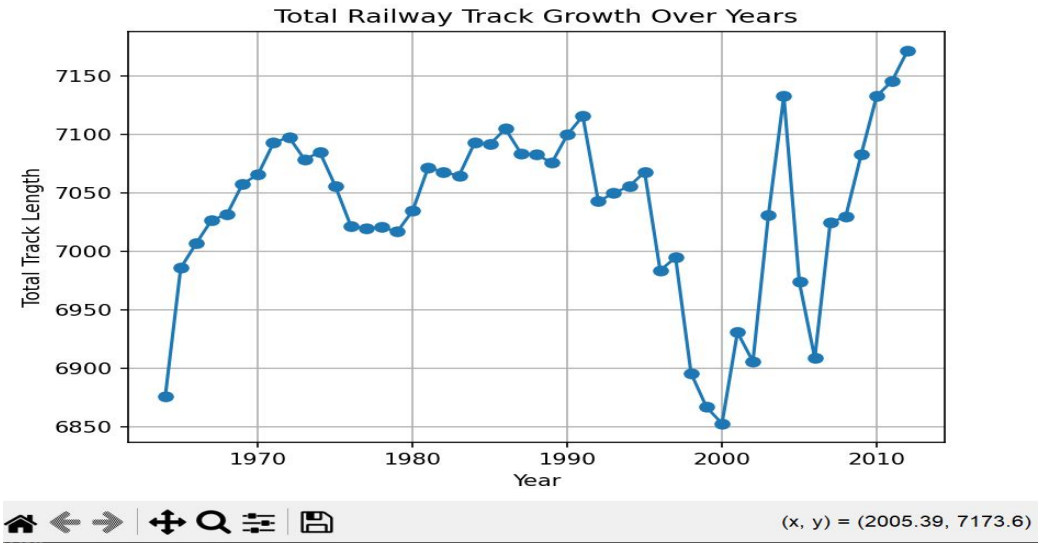
```
# 2.3 Plot line graph (trend visualization)
plt.plot(data['Year'], data['Total'], marker='o')

# add chart title
plt.title('Total Railway Track Growth Over Years')

# X and Y axis labels
plt.xlabel('Year')
plt.ylabel('Total Track Length')

# 2.4 Display graph and observe trend (increase/decrease)
plt.grid(True)
plt.show()
```

- **plt.plot()**: This is the "Artist." It maps the years on the bottom and the track length on the side, drawing a line to show the trend.
- **marker='o'**: This adds dots to each year's data point, making it easier for the client to see exact years.
- **plt.grid(True)**: This adds a background grid (like graph paper), which helps the client trace the numbers accurately.



The graph shows a clear upward slope.(increase)

Scenario 3: Filtering + Bar Char

Filter:
First, we "zoom in" on the data. We only want to show the client what has happened in the last 25 years.

```
#  Scenario-3--Filtering & Bar Chart
#-----------------------------------------------------------------------------------------
# 3.1 Filter the dataset for years after 2000
df['Year'] = df['Year'].str[:4].astype(int)
recent_df = df[df['Year'] > 2000]

# 3.2 Select Broad Gauge, Metre Gauge, and Narrow Gauge
gauge_data = recent_df[['Year', 'Broad Gauge', 'Metre Gauge', 'Narrow Gauge']].copy()
```

Main functions:

a. **df[df['Year'] > 2000]**: This is a **Filter**. It hides all the old historical data so the client can focus on modern infrastructure.
b. **astype(int)**: This converts the text year into a whole number so the filter can mathematically understand "greater than 2000."

The Comparison Chart:

Instead of a line, we use bars to show the "Market Share" of each gauge type.

```
# 3.3 Plot a grouped bar chart comparing all three gauges
x = range(len(gauge_data['Year']))

plt.bar(x, gauge_data['Broad Gauge'], width=0.3, label='Broad Gauge')
plt.bar([i + 0.3 for i in x], gauge_data['Metre Gauge'], width=0.3, label='Metre Gauge')
plt.bar([i + 0.6 for i in x], gauge_data['Narrow Gauge'], width=0.3, label='Narrow Gauge')

# 3.4 Add legend and proper labels
plt.xticks([i + 0.3 for i in x], gauge_data['Year'], rotation=45)
plt.xlabel("Year")
plt.ylabel("Number of Tracks")
plt.title("Railway Gauge Comparison After 2000")
plt.legend()

plt.tight_layout()
plt.show()
```

plt.bar(): This is the **Comparison Tool**. It creates vertical blocks. The higher the block, the more tracks exist for that gauge.
width=0.2: This controls how "skinny" the bars are so we can fit three of them side-by-side for every year without them overlapping.

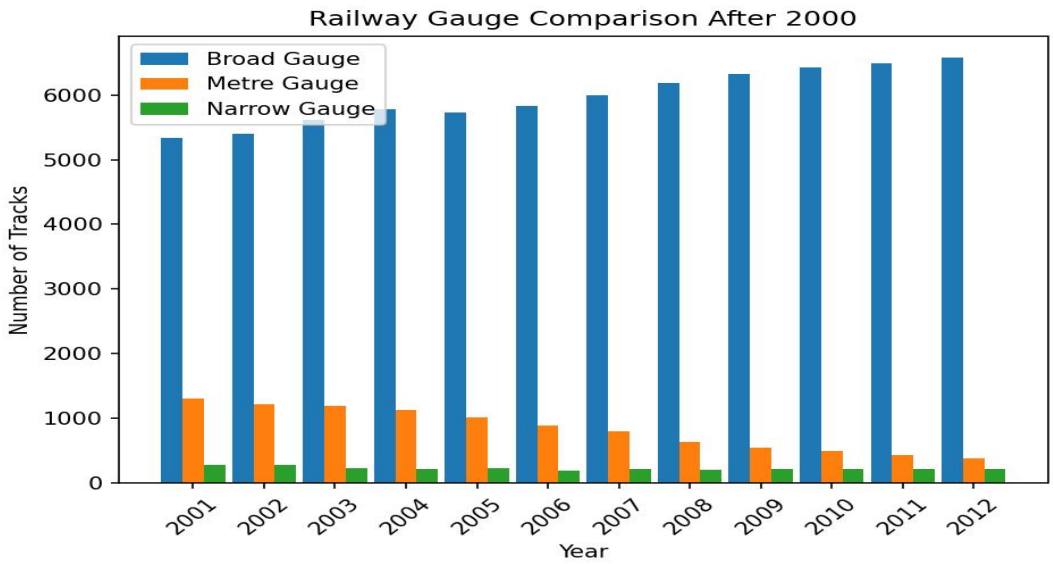
Labels:

```
# 3.4 Add legend and proper labels
plt.xticks([i + 0.3 for i in x], gauge_data['Year'], rotation=45)
plt.xlabel("Year")
plt.ylabel("Number of Tracks")
plt.title("Railway Gauge Comparison After 2000")
plt.legend()

plt.tight_layout()
plt.show()
#plt.savefig("filter&bar.png")

# 3.5 Identify which gauge dominates in recent years
print("Broad Gauge dominates in recent years")
```

plt.legend(): This is the **Key**. It tells the client: "Blue is Broad, Orange is Metre." Without this, the chart is just a bunch of colored sticks.
plt.xticks(): This places the actual Year numbers at the bottom of the bars and rotates them 45 degrees so they don't crash into each other.
Output:



Scenario 4: Feature Engineering + Pie Chart

To analyze the contribution of different railway gauges (Broad, Metre, Narrow) and visualize their percentage share using a pie chart.

Methodology

1. The dataset was loaded from a CSV file using Pandas.
2. The total length of each gauge type was calculated by summing values across all years.
3. A new data structure (Pandas Series) was created to store these totals.
4. A pie chart was plotted using Matplotlib to represent the contribution of each gauge.
5. Percentage labels were added using the autopct parameter for better readability.

Key Functions Used:
sum(): Calculates total of each gauge across all years.This creates a Series storing total values.

```
# 4.1 Calculate total sum of each gauge across all years
totals = df[['Broad Gauge','Metre Gauge','Narrow Gauge']].sum()
```

plt.pie(): Used to create a pie chart ,from Matplotlib.Shows percentage contribution of each gauge.

```
plt.pie(
    totals,
    labels=totals.index,
# 4.4 Add percentage labels (autopct)
    autopct='%1.1f%%',

    startangle=140
)
```

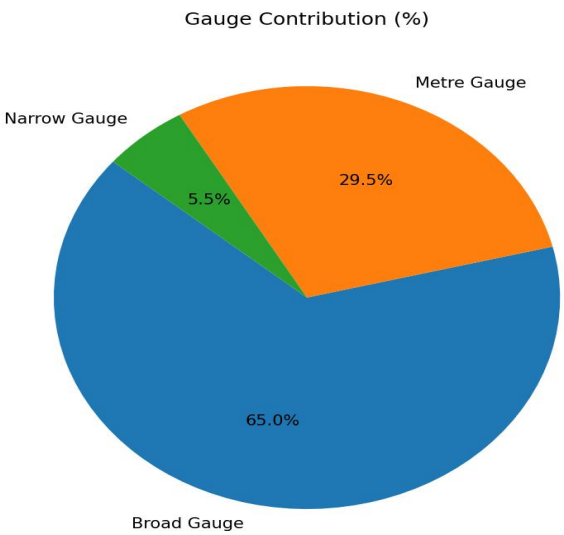
autopct: Displays percentage values on pie chart
Idxmax(): Finds which gauge has highest contribution.

```
# 4.5 Interpret which gauge contributes the most
print("Highest contributing gauge:", totals.idxmax())
```

Result / Observation

- Broad Gauge occupies the largest portion of the pie chart.
- Metre Gauge contributes moderately.
- Narrow Gauge has the smallest share.

Output



Scenario 5: Advanced Analysis + Multiple Graphs

To perform a detailed analysis of railway gauge trends, growth patterns, and composition over time.

Methodology

1. The dataset was imported using Pandas.

2. New percentage columns were created:
    1. % Broad Gauge
    2. % Metre Gauge
    3. % Narrow Gauge
3. Yearly growth in total track length was calculated using NumPy (np.diff).
4. Visualizations were created using Matplotlib:
   Line graph to show trends of all gauges over time .Stacked bar chart to display gauge composition

5.Key insights were identified:
    1. Year with highest growth
    2. Years where decline occurred in Metre and Narrow gauges

Key Functions Used

Creates new columns for gauge contribution.

```
# 5.1 Create new columns:
#    % Broad Gauge, % Metre Gauge, % Narrow Gauge
df['% Broad Gauge'] = (df['Broad Gauge'] / df['Total']) * 100
df['% Metre Gauge'] = (df['Metre Gauge'] / df['Total']) * 100
df['% Narrow Gauge'] = (df['Narrow Gauge'] / df['Total']) * 100
```

np.diff()

- Used to calculate year-to-year growth , From NumPy

```
# 5.2 Use NumPy (np.diff) to calculate yearly growth of Tota
growth = np.diff(df['Total'])
df['Growth'] = [None] + list(growth)
```

plt.plot(): Used to draw line graph.

```
plt.plot(df['Year'], df['Broad Gauge'], label='Broad Gauge')
plt.plot(df['Year'], df['Metre Gauge'], label='Metre Gauge')
plt.plot(df['Year'], df['Narrow Gauge'], label='Narrow Gauge')
```

plt.bar() : Shows composition of gauges.

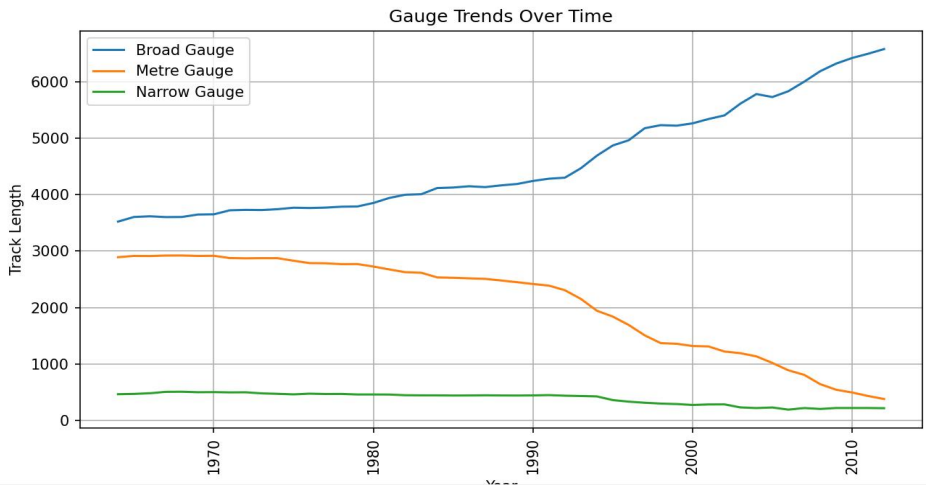
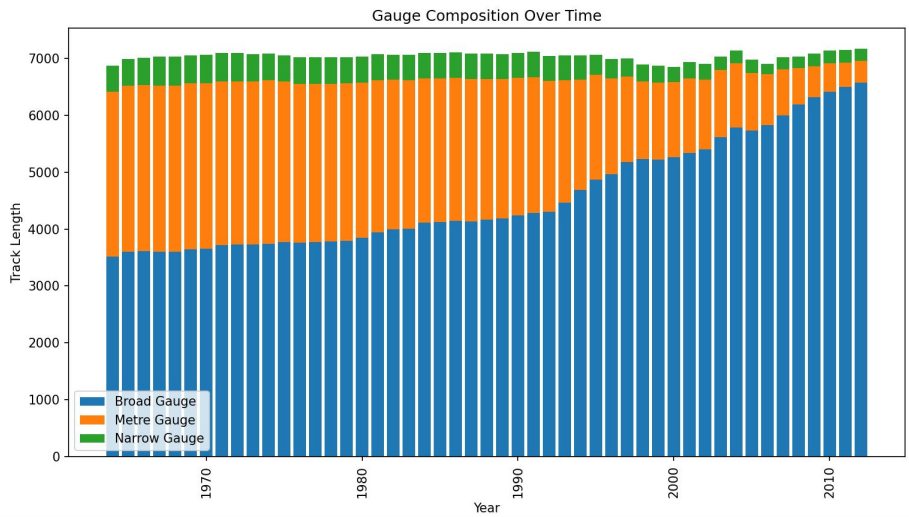
```
plt.bar(df['Year'], df['Broad Gauge'], label='Broad Gauge')
plt.bar(df['Year'], df['Metre Gauge'],
        bottom=df['Broad Gauge'], label='Metre Gauge')
plt.bar(df['Year'], df['Narrow Gauge'],
        bottom=df['Broad Gauge'] + df['Metre Gauge'],
        label='Narrow Gauge')
```

diff(): Detects increase/decrease in individual gauges.

```
# 5.4.2 Highlight decline in any gauge
print("Metre Gauge declining years:",
      df['Year'][df['Metre Gauge'].diff() < 0].tolist())
```

idxmax(): Finds year with highest growth

Result / Observation

- Broad Gauge shows a steady and continuous increase over the years.
- Metre Gauge shows a consistent decline.
- Narrow Gauge also declines significantly.
- Growth in total track length varies across years, with some peak growth periods.

Output